

TETHERLESS: 5-PERSON SPEECH DECK

Decentralized End-to-End Encrypted Messaging Architecture • Comprehensive Technical Presentation

Format: 25 Widescreen Slides (6 Parts)

Target Duration: 20 Minutes 30 Seconds + Q&A

System Status: Pre-1.0 Verified

Slide Deck: docs/tetherless_presentation.pptx

1. Team Roster & Slide Distribution Matrix

Executive Bookend Model

Speaker	Role / Technical Persona	Assigned Slides	Core Technical Topics	Time Target
SPEAKER 1	Systems Architect & Team Lead (Emcee)	Slides 1–5, 24–25 (7 slides)	Vision, Hostile Relay Axiom, Codebase Topology, Post-1.0 Horizons, Conclusion	4:30 min
SPEAKER 2	Cryptography & Wire Protocol Engineer	Slides 6–9 (4 slides)	AES-256-GCM, DH-2048 (RFC 3526), Deterministic Nonces, Binary Codec vs RCE	4:00 min
SPEAKER 3	Security Analyst & Threat Modeling Lead	Slides 10–13 (4 slides)	Threat Matrix T1–T6, TOFU Identity Binding, Safety Numbers, Honest Limitations	3:45 min
SPEAKER 4	Distributed Systems & Relay Lead	Slides 14–18 (5 slides)	Dumb Relay Queues, CAS Reconnect, -PreleaseBuild Hardening, Desktop Swing & WAL	4:15 min
SPEAKER 5	Mobile Platforms & QA / Release Lead	Slides 19–23 (5 slides)	Android Room + SQLCipher + KeyStore, Update Banner, 13 Adversarial Tests, MVP Tickets	4:00 min

Spk 1: Intro (4:30)

Spk 2: Crypto (4:00)

Spk 3: Threats (3:45)

Spk 4: Systems (4:15)

Spk 5: Mobile & QA (4:00)

2. Slide-by-Slide Delivery Scripts, Visual Cues & Verbal Handoffs

Slides 1 through 25

SLIDE 1

Title Slide: TETHERLESS E2EE Messaging Architecture

SPEAKER 1

0:45 min

Visual on Screen: Dark navy title slide with cyan glowing typography, status badge "Pre-1.0 Verified • 6-Part Presentation • 25 Slides".

"Good morning, everyone. I'm **[Speaker 1 Name]**, Lead Architect for project **Tetherless**, and together with my engineering team, we are excited to present our architecture and technical specification for decentralized, end-to-end encrypted messaging.

In today's communication ecosystem, almost every so-called 'secure' platform demands that you trust their intermediate infrastructure. At Tetherless, we start from an entirely different foundational axiom: **the relay server is assumed actively hostile**.

Today, we will walk you through how we designed, implemented, and mathematically verified a zero-knowledge communication stack that guarantees confidentiality, integrity, and authenticity across desktop and mobile platforms."

Stage Direction: Stand center-stage. Speak with calm authority, establishing an engineering-first standard.

SLIDE 2 Agenda & Presentation Roadmap

SPEAKER 1

0:45 min

Visual on Screen: 6-card grid outlining Parts 1 through 6 from Architecture down to Verification and Roadmap.

"Our presentation is organized into six structured technical deep dives.

I will begin with our **Executive Overview and Multi-Module Architecture**. Next, **[Speaker 2 Name]** will take the floor to detail our **Cryptographic Engine and Custom Binary Wire Protocol**. **[Speaker 3 Name]** will step through our **Formal Threat Model and Identity Architecture**, proving how we defeat active adversaries. **[Speaker 4 Name]** will explore our **Non-Blocking Relay Server and Desktop Client Platform**. And **[Speaker 5 Name]** will present our **Android Implementation, our 13-scenario Adversarial Test Suite, and our immediate MVP Release Roadmap**.

Finally, I'll return to outline our post-v1.0 horizons and open the floor for your questions. Let's dive into Part 1."

SLIDE 3 Section Divider: Executive Overview & System Architecture

SPEAKER 1

0:20 min

Visual on Screen: Large glowing section card for Part 1 with 3 core bullets.

"Part 1 lays down our core philosophy and code topology. Before writing a single line of cryptography, we had to redefine the fundamental trust relationship between clients and the network."

SLIDE 4 The Core Premise: The Relay is Assumed Hostile

SPEAKER 1

1:20 min

Visual on Screen: Dual cards: Left card (Zero-Trust Relay Principle) in Cyan; Right card (Guarantees & Non-Goals) in Teal/Amber.

"Here is our core premise: **We do not treat the relay as a trusted partner**. In our threat model, the relay server is operated by an active adversary. It is assumed compromised, actively snooping, tampering with packets, or monitored by a hostile third party.

To survive this environment, Tetherless enforces four strict rules:

First, **zero plaintext knowledge**. The relay never possesses private keys, never performs decryption, and handles message payloads strictly as opaque byte arrays.

Second, **context hiding**. Reply quotes, parent message IDs, and conversation threading are packaged *inside* the encrypted envelope—the relay sees only ciphertext.

Third, **dual-layer protection**. We apply pinned TLS 1.3 at the transport layer for link integrity and ISP traffic-masking, but we *never* rely on TLS as a substitute for application-layer E2EE.

On the right card, notice our engineering honesty: We guarantee message confidentiality, tamper evidence, and replay defense. But we explicitly document our non-goals for v1.0: the relay necessarily observes connection metadata—who connects, when, and packet byte counts. We believe security claims are meaningless without explicitly documenting their boundaries."

Stage Direction: Gesture toward the right-hand card when emphasizing honest limitations.

Visual on Screen: 4 horizontal cards: core-shared (Cyan), chat-server (Rose), chat-desktop (Teal), chat-mobile (Green).

"To enforce these boundaries physically in the build system, Tetherless is structured into four decoupled Gradle modules.

At the center is **core-shared**. This is a platform-neutral library containing our cryptographic primitives, wire codec, message models, and handshake state machine. Crucially, it targets Java 8 bytecode compatibility so it runs identically on both modern desktop JVMs and Android devices, preventing cryptographic logic drift.

Below that is **chat-server**. This is a deliberately dumb, high-throughput forwarder. It does not import database libraries, holds no client keys, and exists solely to route encrypted frames via non-blocking queues.

On the client tier, **chat-desktop** is our Java Swing desktop application featuring SQLite in WAL mode, PBKDF2 at-rest encryption, and standalone Windows .msi packaging.

And finally, **chat-mobile** delivers our native Android client with Room, SQLCipher whole-file encryption, and hardware-backed keys."

Verbal Handoff to Speaker 2: "Now, to unpack the mathematical core that powers core-shared, I'll hand the mic to our Cryptographic & Protocol Engineer, [Speaker 2 Name]."

Visual on Screen: Section divider for Part 2 with 3 bullets highlighting algorithms, nonces, and codec.

"Thank you, [Speaker 1 Name].

In Part 2, we will look under the hood at the mathematical algorithms, key derivation schemes, and custom binary framing that secure every byte transmitted across Tetherless."

Visual on Screen: Left card: Cipher Suite & Key Agreement; Right card: Key Derivation & Storage Security.

"Our cryptographic architecture relies on standardized, constant-time primitives rather than custom inventions.

For payload confidentiality and authentication, we use **AES-256-GCM** with a 96-bit nonce and a 128-bit authentication tag, providing authenticated encryption with associated data.

For session key agreement, we implement **RFC 3526 MODP Group 14 Diffie-Hellman**—a 2048-bit safe prime with generator $g=2$. By switching to standardized static parameters instead of computing runtime primes, we cut client key generation from over 30 seconds down to under 50 milliseconds. Crucially, we enforce strict inbound public key validation: we verify that $1 < y < p-1$ and $y^q \bmod p == 1$, which completely neutralizes small-subgroup confinement attacks.

For identities, each device generates an **RSA-2048** keypair used to digitally sign handshakes and messages via SHA256withRSA over canonically ordered bytes.

On key derivation: Session keys are expanded using **HKDF-SHA256**. Notice the bullet on **DH Normalization**: In early testing, we uncovered an intermittent 1-in-256 handshake failure between desktop Java and Android. The culprit was BigInteger leading-zero truncation. We implemented left-padding normalization on the shared secret prior to HKDF extraction, permanently resolving cross-platform interop."

Visual on Screen: Left card (Nonce Reuse Hazard) in Rose; Right card (Deterministic Nonce) in Green with code buffer highlight: [Direction: 4 bytes] || [Monotonic Counter: 8 bytes].

"If there is one cardinal rule in modern symmetric cryptography, it is: **never reuse a nonce under the same key in AES-GCM**. Doing so completely destroys confidentiality via keystream XOR cancellation and allows polynomial forgery of the authentication tag.

Many implementations rely on random 96-bit nonces, but over long-lived sessions, the Birthday Paradox creates an unacceptable collision risk. Worse, during our early prototyping, we caught a subtle bidirectional trap: if Alice and Bob both start message counters at zero, their very first outgoing messages will collide under the shared key!

On the right card, you see our solution: a deterministic **12-byte (96-bit) nonce structure**. We dedicate the first **4 bytes to a Direction identifier** and the trailing **8 bytes to a Monotonic Counter**. The direction identifier is computed deterministically by comparing both parties' cryptographic Peer IDs lexicographically. The party with the lower alphanumeric ID transmits on 0x00000000, while the higher transmits on 0x00000001.

Furthermore, we enforce a strict send budget of **100,000 messages per session**. Exceeding this triggers mandatory session rekeying before counter wraparound is even remotely possible. This scheme was empirically tested over 100,000 continuous transmissions with zero duplicate nonces."

Stage Direction: Emphasize the critical danger of GCM nonce reuse; this is a major technical flex for the team.

Visual on Screen: Left card (Java Deserialization Hazard) in Rose; Right card (Binary Frame Codec) in Cyan with framing format.

"One of our most critical architectural decisions was the complete elimination of Java native serialization.

Java's `ObjectInputStream.readObject()` is notorious for Remote Code Execution vulnerabilities via classpath gadget chains. Furthermore, `ObjectOutputStream` maintains an internal object table that leaks memory unless manually reset, and untrusted streams can declare multi-gigabyte array allocations, causing immediate remote Denial of Service.

To permanently close these vectors, we stripped every `Serializable` interface from our codebase and engineered a custom **Length-Prefixed Binary Frame Codec**.

Look at the framing specification on the right: Every message starts with a 4-byte length prefix and a 1-byte version header. Most importantly, **we validate strict per-field size caps before allocating buffer memory**. Peer IDs cannot exceed 128 bytes, payloads are capped at 64 kilobytes, signatures at 512 bytes, and the total frame cannot exceed 1 megabyte.

We subjected this binary codec to automated fuzz testing with over 20,000 random bit-flips and mutations, verifying that malformed frames trigger typed protocol exceptions and never crash the process."

Verbal Handoff to Speaker 3: "To show how these cryptographic guarantees map into our comprehensive threat model, I'll pass the mic to our Security Analyst, [Speaker 3 Name]."

Visual on Screen: Section divider for Part 3 with 3 key threat matrix overview bullets.

"Thank you, [Speaker 2 Name].

A security system is only as good as the threat model it is measured against. In Part 3, we present our formal attack matrix, our identity verification mechanisms, and an honest account of our threat boundaries."

Visual on Screen: 6-box matrix detailing Attacks T1 through T6 with color-coded severity accents.

"Our security architecture was built to withstand six explicit attack vectors, designated T1 through T6.

T1 is Passive Relay Sniffing: The relay operator taps the wire. Mitigated by AES-256-GCM. We verified this with a live packet tap during our End-to-End Exchange tests, mathematically asserting zero plaintext leakage.

T2 is Active Man-in-the-Middle by the Relay: The relay attempts to inject its own Diffie-Hellman keys during key agreement. Mitigated because every DH parameter is digitally signed with the sender's long-term RSA key, and peer addresses are mathematically bound to the key hash.

T3 is Intermediate Network Interception: Mitigated by our TLS 1.3 transport layer with client-side certificate pinning, protecting packet timing and metadata from external ISPs.

T4 is Replay and Packet Reordering: An adversary captures a valid encrypted frame and replays it later. Mitigated by a monotonic sliding counter window with a floor of `highest_seen - 1024`, combined with a strict 5-minute timestamp skew rejection.

T5 is Malicious Peer Payloads: Mitigated by the custom binary codec you just saw, enforcing pre-allocation bounds.

And **T6 is Device Compromise at Rest:** If the device is seized or stolen while powered off, databases are encrypted using SQLCipher on Android and PBKDF2-HMAC-SHA256 on Desktop. Keys are never written to disk in plaintext."

Visual on Screen: Left card: Cryptographic Peer Identity; Right card: Trust-on-First-Use & Safety Numbers.

"In decentralized systems, how do users verify who they are talking to without a central certificate authority?"

In Tetherless, identity is mathematical. A **Peer ID** is strictly the first 16 bytes of the SHA-256 hash of the user's public key, rendered as 32 hexadecimal characters. It is immutable. Display names are treated as untrusted metadata and sanitized of bidirectional Unicode overrides to prevent visual spoofing.

When two users first connect, we implement **Trust-on-First-Use (TOFU)**. The first public key received for a peer is permanently pinned in the local database. If an active relay or attacker attempts to swap that key in a subsequent session, the client hard-blocks communication and flashes a red KEY_CHANGED warning banner.

To defeat MITM attacks even on the first contact, we generate **Safety Numbers**: 12 groups of 5 decimal digits computed over both public keys sorted lexicographically: $\text{SHA-256}(\min(KA, KB) \parallel \max(KA, KB))$. Because of this sorting, both Alice and Bob compute the exact same digits. They can compare them out-of-band over a phone call or in person to verify absolute cryptographic authenticity."

Visual on Screen: Left card (Architectural Non-Guarantees) in Amber; Right card (Cryptographic Scope Boundaries) in Rose.

"No security review is complete without discussing limitations. We believe in total transparency regarding what Tetherless v1.0 does and does not protect against.

On the left: **Relay metadata observation**. The relay sees the social graph: who connects, message frequency, and packet lengths. We do not yet implement dummy traffic padding. Furthermore, a single central relay is a single point of failure for availability until we federate.

On the right: **Forward secrecy scope**. Session keys are ephemeral, but forward secrecy is bound by session volume—up to 100,000 messages—rather than per-message ratchet steps. Double Ratchet is slated for post-1.0. Finally, at-rest encryption protects cold data, but an active compromise of live process memory on an unlocked device is outside our threat model."

Verbal Handoff to Speaker 4: "Now, let's examine how the relay server handles high throughput while maintaining this zero-knowledge boundary. I'll hand over to **[Speaker 4 Name]**."

Visual on Screen: Section divider for Part 4 with 3 relay engineering bullets.

"Thank you, **[Speaker 3 Name]**."

In Part 4, we examine the relay server architecture—how we built a high-throughput, non-blocking routing engine that is deliberately dumb and hardened for production deployment."

SLIDE 15 Dumb Relay: Non-Blocking Ciphertext Routing

SPEAKER 4

1:25 min

Visual on Screen: Left card: Non-Blocking Per-Client Writer Queues; Right card: Concurrency & Connection Lifecycle.

"The primary engineering challenge in a real-time relay is preventing slow or malicious clients from degrading the system for everyone else.

In Tetherless, we implement a **dedicated non-blocking writer queue pattern**. Each connected client session maintains an in-memory `ArrayBlockingQueue<byte[]>` with a capacity of 256 frames, serviced by its own background writer thread. When a message arrives, the inbound socket handler parses only the 4-byte length and receiver ID, deposits the raw ciphertext frame into the recipient's queue, and returns immediately.

This achieves **zero head-of-line blocking**: A slow cellular client on high packet loss cannot stall the sender or consume shared server threads. If a client queue fills to capacity, the server closes the offending connection with an ERROR frame rather than deadlocking.

On the right card, look at our connection lifecycle handling: In mobile networks, devices disconnect and reconnect frequently. To prevent race conditions where a dying socket evicts a newly established socket, our ClientRegistry uses atomic Compare-And-Swap operations: `remove(clientId, thisSession)`. We maintain a 30-second heartbeat PING interval and a strict 90-second socket read timeout, ensuring abandoned half-open TCP connections are reaped immediately. We stress-tested this engine with 10 concurrent senders blasting 1,000 messages simultaneously to a single recipient—10,000 messages delivered with zero loss and zero reordering."

SLIDE 16 Operational Hardening & Release Gating

SPEAKER 4

1:10 min

Visual on Screen: Left card (Release Gate Enforcement) in Amber; Right card (Container Sandboxing) in Green.

"Engineering secure code is futile if operational deployment is sloppy.

In development, we use a self-signed keystore for local testing. In many commercial breaches, developers accidentally ship these development credentials to production. In Tetherless, we solved this with an **enforced -PreleaseBuild Gradle release gate**. When building a production distribution, the build system physically strips `dev-keystore.p12` from the JAR and Docker image. The server binary literally refuses to start unless an external, production keystore is supplied. To prevent secrets from appearing in process listings or docker inspect, passwords can be mounted via `KEYSTORE_PASSWORD_FILE`.

For containerization, our Docker image runs as an unprivileged user—`tetherless:tetherless`—with a **read-only root filesystem and ALL Linux capabilities dropped**. Finally, Prometheus metrics on port + 1 are strictly bound to loopback 127.0.0.1, guaranteeing unauthenticated operational telemetry is never exposed to the public internet."

SLIDE 17 Section Divider: Client Implementations: Desktop & Mobile

SPEAKER 4

0:15 min

Visual on Screen: Section divider for Part 5 with 3 client overview bullets.

"Now let's move into Part 5 and examine how these server and protocol contracts are realized across our client applications, starting with the Desktop client."

Visual on Screen: Left card: UI & Threading Model; Right card: Persistence & Packaging.

"The desktop client is built with Java Swing, but engineered to modern responsive standards.

First, **strict Event Dispatch Thread (EDT) decoupling**. All network sockets, cryptographic calculations, and database reads run on background worker pools. UI components receive updates exclusively through `SwingUtilities.invokeLater`, ensuring a silky-smooth, freeze-free user interface. Our `ConnectionManager` executes an exponential backoff state machine with full jitter, gracefully recovering from network drops between 1 and 60 seconds. We also eliminated a legacy security flaw: identity key generation now occurs 100% in-process via JCE, rather than shelling out to the CLI keytool which leaked passphrases in the OS process table.

For persistence, we use SQLite configured in **WAL (Write-Ahead Logging) mode** with a single-writer connection, preventing database lock contention under rapid messaging. Message history is encrypted at rest using PBKDF2-derived keys.

And for deployment, we package the desktop app via `jpackage` into a self-contained Windows .msi that bundles a trimmed modular JRE—users can install and run Tetherless without having Java installed on their machine!"

Verbal Handoff to Speaker 5: "Now, to present our Android architecture, our update strategy, and our verification suite, I'll pass the mic to [Speaker 5 Name]."

Visual on Screen: Left card: Android Architecture & Storage; Right card: Service Lifecycle & Core Shared Engine.

"Thank you, [Speaker 4 Name].

On mobile, our design mandate was two-fold: deliver modern reactive Android architecture, and maximize hardware-level cryptographic isolation.

On the left: The app is built on Google's modern Jetpack stack—Room ORM, ViewModels, LiveData reactive streams, and ListAdapter with DiffUtil. Unlike desktop column encryption, **Android uses SQLCipher for whole-file database encryption**. Every table, participant record, and timestamp is encrypted before hitting flash storage. Crucially, the 256-bit database key is wrapped using AES-GCM and stored inside the **hardware-backed AndroidKeyStore**. The raw key exists only in transient memory and is zeroed out on teardown.

On the right: Mobile OSs aggressively kill background sockets. We implemented a persistent **Foreground ChatService** with a low-priority notification, allowing the connection to survive Android Doze mode and memory reclamation. Notice that both Desktop and Mobile delegate all crypto and wire handling to core-shared—giving us 100% business logic parity across platforms."

SLIDE 20 Update Delivery & Maintenance Strategy

SPEAKER 5

0:50 min

Visual on Screen: Left card: Update Checker Architecture (CLIENT-DESKTOP-08); Right card: Security & Privacy Policy.

"Maintaining desktop software requires an update notification pipeline that respects user privacy.

Under ticket **CLIENT-DESKTOP-08**, our desktop client queries the GitHub Releases API asynchronously on startup, comparing the latest release tag against BuildInfo.version stamped during CI. If an update is found, it presents a clean, non-intrusive banner in the chat window with a link to download the new installer. If the network is down or rate limits hit, it fails completely silently.

Notice our privacy policies on the right: **We refuse to auto-execute downloaded binaries**. Since the pre-1.0 MVP is not yet signed with a commercial EV certificate, auto-executing code is dangerous. Furthermore, the update check transmits **zero user telemetry, zero device IDs, and zero metadata**, and privacy-conscious users can completely disable update checks with a single configuration flag."

SLIDE 21 Section Divider: Verification, Testing & Production Roadmap

SPEAKER 5

0:20 min

Visual on Screen: Section divider for Part 6 with 3 testing and roadmap bullets.

"Now we enter our final section: Part 6. Here we move from architecture on paper to empirical proof: our automated adversarial attack suites, our static analysis gates, and our roadmap to shipping v1.0."

SLIDE 22 Verification Suite & Adversarial Testing

SPEAKER 5

1:20 min

Visual on Screen: Left card: AdversarialRelayTest (13 Attack Scenarios) in Rose; Right card: End-to-End & Golden Conformance in Green.

"We do not assume our security works; we attack our own code in automated test suites.

On the left is our primary adversarial test: **AdversarialRelayTest**, which injects 13 real-world active attacks: When a malicious relay attempts key substitution during the DH handshake, the handshake aborts immediately. When an attacker substitutes the public key in a HELLO frame, it is rejected because the Peer ID does not match the key hash. When we flip a single bit in the ciphertext payload, GCM authentication tag verification fails and the frame is dropped. When a captured frame is replayed, the sliding counter window flags it as a duplicate and discards it.

On the right, look at **EndToEndExchangeTest**. We spin up a real relay and two live clients, attach an external network wire tap, and transmit 200 real messages. The test asserts mathematically that not a single substring of plaintext ever appears on the wire. We also validate **17 frozen golden wire vectors** across both the JVM and Android emulator to guarantee zero codec divergence, and our static analysis gate runs SpotBugs with find-sec-bugs with **zero high-severity warnings**."

Visual on Screen: 6-ticket backlog matrix detailing CLIENT-DESKTOP-07 through REL-03 & INTEG-04.

"Here is our exact sprint backlog leading directly to the production Server and Windows Desktop MVP release:

1. **CLIENT-DESKTOP-07 (Currently Active)**: Expanding desktop unit test coverage across the repository, connection manager, and cryptographic pipelines.
2. **INTEG-01**: A headless automated multi-client integration test harness validating live message routing.
3. **BUILD-04**: Our GitHub Actions CI pipeline running clean multi-platform builds, SpotBugs, unit tests, and integration suites on every commit.
4. **REL-01**: Formal security audit, log scrubbing review to ensure zero key leakage, and publication of our threat model in docs/security.md.
5. **CLIENT-DESKTOP-08**: The in-app update notification banner we discussed earlier.
6. And **REL-03 & INTEG-04**: Final manual QA matrix execution on Windows 10 and 11, accompanied by our complete deployment documentation overhaul."

Verbal Handoff to Speaker 1: "To close out our presentation and share our future architectural horizons, I will pass the floor back to our Lead Architect, [Speaker 1 Name]."

Visual on Screen: Left card: Cryptographic Upgrades (Double Ratchet, Curve25519); Right card: Decentralization & Topology (Federation, MLS).

"Thank you, [Speaker 5 Name].

Once our v1.0 MVP is deployed to production, our architectural roadmap expands in two exciting dimensions.

In cryptography: Ticket **FUTURE-01** introduces Signal's **Double Ratchet protocol**, evolving our session-bound forward secrecy into per-message forward secrecy with post-compromise self-healing. Under **FUTURE-02**, we will transition from classical DH-2048 and RSA-2048 to **Curve25519**—using X25519 for key exchange and Ed25519 for signatures—slashing handshake sizes and accelerating mobile battery efficiency.

In network topology: Ticket **FUTURE-04** will transition Tetherless from a single relay into a **federated, peer-to-peer network** using DHT-based peer discovery, eliminating the single relay as an availability bottleneck. And **FUTURE-05** will bring decentralized group messaging through the **Messaging Layer Security (MLS)** standard.

Tetherless v1.0 is engineered specifically so that these future protocols can be dropped into core-shared without rewriting client UI or persistence layers."

Visual on Screen: Full-width summary card: 5 Key Takeaways & Production Readiness.

"To summarize what we have achieved with Tetherless:

1. **An uncompromising security core:** AES-256-GCM authenticated encryption, signed Diffie-Hellman, deterministic nonces, and a custom binary frame codec that permanently eliminates Java deserialization RCE.
2. **A provable zero-knowledge relay:** Validated by automated wire taps proving zero plaintext ever touches intermediate servers.
3. **Cross-platform parity:** A shared cryptographic engine guaranteeing flawless interop between Windows Desktop and Android.
4. **Production-hardened operations:** Docker sandboxing with dropped capabilities, loopback telemetry, and release gates that prevent shipping development certificates.
5. **A clear, actionable path to MVP:** A prioritized ticket backlog delivering a shippable, pre-1.0 verified product.

On behalf of our entire engineering team—**[Speaker 2]**, **[Speaker 3]**, **[Speaker 4]**, **[Speaker 5]**, and myself—thank you for your time. We would now be delighted to answer any questions."

Stage Direction: Step back into alignment with the entire team. Open arms warmly to invite audience questions.

3. Team Q&A Defense Strategy & Question Routing

Speaker 1 Moderates & Dispatches to Specialists

Question Category	Primary	Backup	Key Technical Talking Point
Why not use Double Ratchet in v1.0?	SPEAKER 2	Speaker 1	"Double Ratchet introduces significant out-of-order ratchet state and key management overhead. For v1.0 direct messaging, RFC 3526 DH-2048 with a 100,000-message session budget provides rigorous forward secrecy while keeping the state machine verifiable and stable. Double Ratchet is queued as FUTURE-01."
Why raw TCP sockets instead of WebSockets or gRPC?	SPEAKER 4	Speaker 2	"WebSockets and gRPC introduce extensive HTTP/2 framing, framing headers, and third-party library dependencies. Raw TCP sockets with our length-prefixed binary codec give us minimal binary overhead, zero external framework CVE surfaces, and absolute control over memory allocation bounds."
What prevents an active relay from dropping or altering messages?	SPEAKER 3	Speaker 5	"If the relay alters even one bit of ciphertext, GCM authentication tag verification fails instantly and the message is discarded. If the relay drops frames, the recipient notices counter gaps or timeout disconnects. While a relay can deny service, it can never tamper without detection."
How does the relay scale with thousands of slow connections?	SPEAKER 4	Speaker 1	"Our dedicated ArrayBlockingQueue<byte[]> pattern ensures that a slow or lagging client only fills its own 256-frame queue. Inbound worker threads deposit frames and return in sub-millisecond time. If a client queue overflows, only that slow client connection is terminated—zero head-of-line blocking."
How do you ensure Android and Desktop cryptography stay in sync?	SPEAKER 5	Speaker 2	"Both applications import the exact same core-shared library compiled to Java 8 bytecode. Furthermore, our CI runs ProtocolVectors containing 17 frozen wire-format vectors across both the JVM and an Android emulator to assert byte-for-byte serialization parity."
How does the binary codec prevent Remote Code Execution?	SPEAKER 2	Speaker 4	"All classes stripped of Serializable. We use explicit length-prefixed DataInputStream/DataOutputStream decoding. Per-field caps (IDs ≤ 128B, Payload ≤ 64KB, Sig ≤ 512B) are checked before allocating heap memory, completely closing gadget chain deserialization."
How are identities verified without a central CA?	SPEAKER 3	Speaker 2	"Identity is cryptographic: Peer ID is SHA-256(PublicKey)[0..16]. Handshakes are digitally signed with RSA-2048. Trust-on-First-Use pins keys locally; subsequent changes trigger KEY_CHANGED alerts. Symmetric 60-digit safety numbers enable out-of-band cross-verification."
What prevents accidental deployment with development certificates?	SPEAKER 4	Speaker 5	"The Gradle -PreleaseBuild gate physically removes dev-keystore.p12 from the build output. The server binary checks keystore existence on boot and aborts immediately if dev certificates are detected."
How is message history protected if a phone is stolen?	SPEAKER 5	Speaker 3	"On Android, SQLCipher encrypts the entire SQLite file at rest. The 256-bit database key is wrapped using hardware-backed AndroidKeyStore (AES-GCM). Raw keys exist only in transient RAM and cannot be exported or backed up."
What is the long-term plan for decentralization?	SPEAKER 1	Speaker 4	"Post-v1.0 tickets FUTURE-04 and FUTURE-05 federate relays and introduce DHT-based peer discovery. Because core-shared encapsulates the session state machine, we can switch routing layers without modifying UI or storage code."

4. Team Rehearsal & Execution Checklist

5 Ground Rules for Presentation Day

- 1. Strict Handoff Cadence:** Memorize your 1-sentence handoff and pickup lines. Never leave dead air between speakers.
- 2. Stage Positioning:** Stand in sequence: [Speaker 1] [Speaker 2] [Speaker 3] [Speaker 4] [Speaker 5]. Step forward when speaking; step back when handing off.
- 3. Coordinated Slide Clicks:** Speaker 1 (or designated clicker) advances slides using the verbal cue markers indicated in the script.

4. Physical Gestures: Actively point to the left/right slide cards and visual diagrams when speaking about them (e.g., the 12-byte nonce buffer, the threat matrix).

5. Q&A Discipline: Speaker 1 acknowledges questions and delegates to the primary specialist. Never talk over each other during evaluation questions.
